AI-Architect Architecture Report

Project: KIConnect Greenfield

Run ID: 20260828T141205Z-3a0086f8

Status: Accepted — 2026-08-28 14:54 UTC

Final Review Verdict: PASS

Models used: gemini-3.1-flash-lite, OpenAI GPT OSS 120b KI:Inferenz.nrw

Executive Summary

Customers browse, search, add items to carts, and place orders; they expect fast, reliable experiences even during peak campaigns. Staff manage product catalogs, inventory, and order fulfillment; finance processes payments. The platform must support up to 50,000 concurrent users, stay highly available during traffic spikes, and scale automatically while keeping operational costs reasonable.

Requirements & Constraints

Business Goal

Build a new e-commerce platform that can support future growth and high peak traffic.

Problem Statement

We want to build a new e-commerce platform from scratch that can support future growth and high peak traffic.

Users / Stakeholders

- Online customers, customer support, warehouse and inventory staff, finance and payment operations, product management, software engineers, and operations staff.

Functional Requirements

- Browse and search products
- Manage customer accounts
- Manage shopping carts
- Place and process orders
- Handle payments
- Manage inventory and product availability
- View order status and history

Non-Functional Requirements

- Support up to 50,000 concurrent users during peak campaigns
- Maintain high availability during peak periods
- Keep customer-facing interactions responsive
- Allow the system to scale as demand grows

Cloud Provider

AWS

Budget

No fixed monetary budget has been defined. Cost efficiency should be considered.

Recommended Architecture

Pattern

Hybrid Event-Driven Microservices on AWS

Rationale

Combines domain-driven service boundaries with asynchronous event-driven communication to meet high concurrency, scalability, and availability requirements while keeping user-facing interactions low-latency via synchronous APIs. Managed AWS services and serverless compute are used to control costs in the absence of a fixed budget.

Components

ID	Name	Type	Purpose
COMP-001	Frontend UI	UI	Customer-facing web application for browsing, searching, cart management, and checkout.
COMP-002	Load Balancer	service	Distribute incoming HTTP traffic across API Gateway instances and provide health-checking.
COMP-003	API Gateway	API	Expose synchronous REST/GraphQL endpoints for all front-end interactions.
COMP-004	User Account Service	service	Manage customer registration, authentication, profile updates, and password recovery.
COMP-005	Product Catalog Service	service	Provide product information, categories, and availability data to the front-end.
COMP-006	Search Service	service	Execute full-text product searches and filter queries.
COMP-007	Cart Service	service	Maintain a persistent shopping cart per customer across sessions.
COMP-008	Order Service	service	Create, persist, and manage customer orders and provide order history.
COMP-009	Payment Service	service	Process customer payments securely and report outcomes.
COMP-010	Inventory Service	service	Track stock levels and enforce product availability constraints.
COMP-011	Notification Service	service	Send email/SMS notifications for order confirmations, payment receipts, and shipment updates.
COMP-012	Event Bus	queue	Provide asynchronous, decoupled communication between services.
COMP-013	Data Store	database	Persist domain data for all services using DynamoDB.
COMP-014	Cache	database	Accelerate read-heavy product data and reduce DynamoDB load.
COMP-015	Search Index	database	Store searchable product documents for full-text queries.
COMP-016	Email Service	external	Send transactional email notifications (order confirmations, receipts).
COMP-017	SMS Service	external	Send transactional SMS notifications (order updates, alerts).

COMP-001: Frontend UI

A React single-page application delivered via CDN, interacting with the backend through the API Gateway.

Technology

React, AWS CloudFront, HTTPS

Traces to features

FEAT-001, FEAT-010

Justified by
ADR-003, ADR-004

COMP-002: Load Balancer

AWS Application Load Balancer operating in multiple AZs, terminating TLS and forwarding requests to the API Gateway.

Technology
AWS Application Load Balancer

Traces to features
FEAT-008, FEAT-009, FEAT-010, FEAT-011

Justified by
ADR-004

COMP-003: API Gateway

AWS API Gateway routes HTTP requests to the appropriate backend services and handles request validation, throttling, and authentication.

Technology
AWS API Gateway, JWT authentication

Traces to features
FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Justified by
ADR-003, ADR-004

COMP-004: User Account Service

Stateless service handling account CRUD operations, issuing JWTs, and persisting user data in its own DynamoDB table.

Technology
AWS Fargate, Node.js / TypeScript, DynamoDB (User table)

Traces to features
FEAT-002

Justified by
ADR-001, ADR-002, ADR-003

COMP-005: Product Catalog Service

Stateless service exposing product browse APIs; reads product data from its DynamoDB table and writes inventory updates.

Technology
AWS Fargate, Java / Spring Boot, DynamoDB (Product table)

Traces to features
FEAT-001

Justified by
ADR-001, ADR-002

COMP-006: Search Service

Stateless service that forwards search queries to the OpenSearch cluster and returns ranked results.

Technology
AWS Fargate, Python / FastAPI, Amazon OpenSearch Service

Traces to features
FEAT-001

Justified by
ADR-001, ADR-003, ADR-004

COMP-007: Cart Service

Stateless service that stores cart state in DynamoDB; supports add, update, remove, and retrieval operations.

Technology
AWS Fargate, Go, DynamoDB (Cart table)

Traces to features
FEAT-003

Justified by
ADR-001, ADR-002

COMP-008: Order Service

Stateless service that records orders, reserves inventory, emits OrderCreated events, and serves order-history queries for customers.

Technology
AWS Fargate, Java / Quarkus, DynamoDB (Order table)

Traces to features
FEAT-004, FEAT-007, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Justified by
ADR-001, ADR-002, ADR-003, ADR-004

COMP-009: Payment Service

Stateless service that integrates with external payment providers, records transaction results, and publishes PaymentProcessed events.

Technology
AWS Fargate, Node.js, PCI-DSS compliant third-party gateway

Traces to features
FEAT-005

Justified by
ADR-001, ADR-002, ADR-003

COMP-010: Inventory Service

Stateless service that updates inventory counts, prevents overselling, and emits InventoryReserved events.

Technology
AWS Fargate, Python, DynamoDB (Inventory table)

Traces to features
FEAT-006, FEAT-008, FEAT-009

Justified by
ADR-001, ADR-002, ADR-003, ADR-004

COMP-011: Notification Service

Consumes events from the Event Bus and triggers outbound messages via Amazon SES and SNS.

Technology
AWS Lambda, Node.js, Amazon SES, Amazon SNS

Traces to features
FEAT-004, FEAT-005

Justified by
ADR-003

COMP-012: Event Bus

Implemented with Amazon SNS topics and SQS queues; supports at-least-once delivery and fan-out to multiple consumers.

Technology
Amazon SNS, Amazon SQS

Traces to features
FEAT-008, FEAT-009, FEAT-011

Justified by
ADR-003

COMP-013: Data Store

A set of DynamoDB tables, each owned by a single service, providing low-latency, highly available key-value storage.

Technology
Amazon DynamoDB

Traces to features
FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Justified by
ADR-002, ADR-004

COMP-014: Cache

Redis cluster (ElastiCache) used for caching product listings and session data.

Technology
Amazon ElastiCache for Redis

Traces to features
FEAT-001, FEAT-010

Justified by
ADR-004

COMP-015: Search Index

OpenSearch domain containing indexed product data, refreshed from the Product Catalog Service.

Technology
Amazon OpenSearch Service

Traces to features
FEAT-001

Justified by
ADR-002, ADR-004

COMP-016: Email Service

Managed Amazon SES service used by the Notification Service to deliver email messages.

Technology
Amazon Simple Email Service (SES)

Traces to features
FEAT-004, FEAT-005

Justified by
ADR-003

COMP-017: SMS Service

Managed Amazon SNS SMS capability used by the Notification Service to deliver short message alerts.

Technology
Amazon Simple Notification Service (SNS) SMS

Traces to features
FEAT-004, FEAT-005

Justified by
ADR-003

Data Flows

- Frontend UI → Load Balancer: HTTP traffic from browsers
- Load Balancer → API Gateway: Forward HTTP requests
- API Gateway → User Account Service: Account CRUD API calls
- API Gateway → Product Catalog Service: Product browse API calls
- API Gateway → Search Service: Search query API calls
- API Gateway → Cart Service: Cart manipulation API calls
- API Gateway → Order Service: Checkout request API call
- API Gateway → Payment Service: Payment processing API call
- API Gateway → Inventory Service: Inventory availability check API call
- API Gateway → Order Service: Order history request API call
- User Account Service → Data Store: Read/write user data
- Product Catalog Service → Data Store: Read product data
- Search Service → Search Index: Query OpenSearch index
- Cart Service → Data Store: Persist cart state

- Order Service → Data Store: Persist order data
- Payment Service → Data Store: Persist payment transaction
- Inventory Service → Data Store: Update stock levels
- Order Service → Event Bus: Publish OrderCreated event
- Inventory Service → Event Bus: Publish InventoryReserved event
- Payment Service → Event Bus: Publish PaymentProcessed event
- Event Bus → Notification Service: Deliver order and payment events
- Notification Service → Email Service: Send confirmation emails
- Notification Service → SMS Service: Send SMS notifications
- Data Store → Cache: Populate cache entries
- API Gateway → Cache: Retrieve cached product data

Architecture Decision Records

ADR-001: ADR-001: Adopt Domain-Driven Design bounded contexts for service boundaries

Status: accepted

Context

Need to define clear service boundaries for scalability, maintainability, and independent deployment.

Decision

Decompose the system into services aligned with DDD bounded contexts: User Account, Product Catalog, Search, Cart, Order, Payment, Inventory, Notification.

Rationale

DDD aligns services with business capabilities, reduces coupling, and supports independent scaling as required for peak traffic.

Alternatives Considered

- Monolithic application
- Large coarse-grained services

Positive Consequences

- Clear ownership
- Independent scaling
- Team autonomy

Negative Consequences / Trade-offs

- Increased operational complexity
- Need for inter-service communication

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-007, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Related Components: User Account Service, Product Catalog Service, Search Service, Cart Service, Order Service, Payment Service, Inventory Service, Notification Service

Related Decision Topics: TOPIC-1

Evidence: KB-E004, KB-E001

ADR-002: ADR-002: Use service-owned polyglot persistence with

DynamoDB and OpenSearch

Status: accepted

Context

Each service requires its own data store that matches its access patterns and scalability needs.

Decision

Each microservice owns a private DynamoDB table; the Search Service uses Amazon OpenSearch for full-text search.

Rationale

Service-owned stores prevent accidental coupling and enable independent schema evolution; DynamoDB provides horizontal scalability, while OpenSearch satisfies search requirements.

Alternatives Considered

- Single shared relational database
- All services using the same NoSQL store

Positive Consequences

- Independent data model evolution
- Optimized storage per domain
- Reduced cross-service data coupling

Negative Consequences / Trade-offs

- Increased number of managed databases
- Potential data duplication

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Related Components: Data Store, Search Index, User Account Service, Product Catalog Service, Cart Service, Order Service, Payment Service, Inventory Service

Related Decision Topics: TOPIC-2

Evidence: KB-E005, KB-E008

ADR-003: ADR-003: Hybrid integration style – synchronous API for front-end, asynchronous events for internal processes

Status: accepted

Context

User-facing interactions need low latency, while internal workflows benefit from decoupling and scalability.

Decision

Expose REST/GraphQL endpoints via API Gateway for all external calls; use SNS/SQS Event Bus for order, inventory, and payment events between services.

Rationale

Synchronous APIs give immediate feedback to customers; asynchronous events improve throughput and fault tolerance for background processing.

Alternatives Considered

- Fully synchronous request-response across all services
- Fully asynchronous event-driven architecture

Positive Consequences

- Responsive UI
- Scalable backend processing
- Improved fault isolation

Negative Consequences / Trade-offs

- Added complexity of managing both sync and async paths
- Need for eventual consistency handling

Related Features: FEAT-001, FEAT-004, FEAT-005, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Related Components: API Gateway, Event Bus, Order Service, Inventory Service, Payment Service, Notification Service

Related Decision Topics: TOPIC-3

Evidence: KB-E002, KB-E009

ADR-004: ADR-004: Stateless services with AWS autoscaling and managed data stores for high availability

Status: accepted

Context
The platform must handle large traffic spikes without manual capacity planning.

Decision
Deploy services as stateless containers on AWS Fargate behind an Application Load Balancer; rely on DynamoDB, ElastiCache, and OpenSearch which provide built-in autoscaling and multi-AZ replication.

Rationale
Stateless compute enables rapid horizontal scaling; managed data stores remove operational burden and guarantee high availability.

Alternatives Considered

- Self-managed EC2 instances with manual scaling
- Monolithic deployment on a single VM

Positive Consequences

- Automatic scaling to meet demand
- Reduced operational overhead
- Resilience to AZ failures

Negative Consequences / Trade-offs

- Higher per-request cost of managed services
- Less control over underlying infrastructure

Related Features: FEAT-008, FEAT-009, FEAT-010, FEAT-011

Related Components: Load Balancer, API Gateway, Data Store, Cache, Search Index

Related Decision Topics: TOPIC-4

Evidence: KB-E003, KB-E006

ADR-005: ADR-005: Use serverless and managed services to optimize cost

Status: accepted

Context

No fixed monetary budget is defined; cost efficiency must be a primary design driver.

Decision

Leverage AWS managed services (Fargate, DynamoDB, ElastiCache, OpenSearch, SNS/SQS, SES) and serverless compute (Lambda) to ensure that capacity and spend scale proportionally with load, avoiding over-provisioning.

Rationale

Managed services provide pay-as-you-go pricing, automatic scaling, and reduced operational labor, aligning spend with actual usage and satisfying the budget constraint. Evidence shows that managed, stateless architectures reduce capital expenditure and enable cost-effective scaling.

Alternatives Considered

- Self-managed EC2 fleet with manual scaling
- Provisioned capacity on traditional relational databases

Positive Consequences

- Cost aligns with traffic volume
- Lower operational staffing costs
- Built-in scaling and high availability

Negative Consequences / Trade-offs

- Potentially higher per-unit cost at very low utilization
- Vendor lock-in to AWS managed services

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-007, FEAT-008, FEAT-009, FEAT-010, FEAT-011

Related Components: User Account Service, Product Catalog Service, Search Service, Cart Service, Order Service, Payment Service, Inventory Service, Notification Service, Data Store, Cache, Search Index, Email Service, SMS Service

Related Decision Topics: TOPIC-4

Evidence: KB-E006, KB-E010

Evidence / Literature

Curated Knowledge Base Evidence

ID	Source	Page	Excerpt
KB-E001	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	Microservices architecture has emerged as a compelling alternative, offering the promise of enhanced scalability, improved fault tolerance, and greater development agility [1]. By decomposing monolithic applications into smaller, independently deployable services, organizations can achieve better resource utilization,...

ID	Source	Page	Excerpt
KB-E002	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	**b. High Load Scenarios:** Asynchronous patterns (message queues) demonstrate superior throughput and availability [4]. **c. Availability:** Event-driven architectures provide better fault tolerance and system availability under stress [4]. These findings suggest that e-commerce systems should employ hybrid approac...
KB-E003	Rag Database/box1_patterns/architecture_patterns_v2.md	0	**Solution mechanics:** The application is designed to be **stateless** — per AWS, it "does not need knowledge of previous interactions and does not store session information" locally on disk or in memory between requests. All session/user state is offloaded to an external, resilient, multi-zone store (e.g., Amazon El...
KB-E004	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	The reviewed studies reveal several distinct approaches to monolith decomposition, each with specific advantages for e-commerce contexts. **a. Domain-Driven Design (DDD) Approaches:** Multiple studies emphasize Domain-Driven Design as a foundational approach for identifying service boundaries in e-commerce systems. A...
KB-E005	Rag Database/box2_domain/ecommerce_polyglot_persistence_microsoft.md	0	With a domain-driven microservices approach, each service uses the database that fits its data characteristics. Each microservice owns its private data store. This design prevents unintentional coupling between services and supports independent updates and deployments without coordinating changes across the system. #...
KB-E006	Rag Database/box1_patterns/wellarchitected-serverless-applications-lens.pdf	24	Web applications often have demanding requirements to ensure a consistent, secure, and reliable user experience. Workloads which need to scale to thousands or millions of users require provisioning infrastructure for peak loads or sophisticated auto-scaling mechanisms, when available. On-premises workloads require...
KB-E007	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	## 2.2 Monolithic and Microservices Architectures Comparison *(printed p. 478)* In the e-commerce landscape, choosing between microservices and traditional monolithic architectures requires careful consideration of various factors. Monolithic architectures, characterized by their single, tightly coupled codebase. How...
KB-E008	Rag Database/box1_patterns/microservices-on-aws.pdf	10	NoSQL databases have been designed to favor scalability, performance, and availability over the consistency of relational databases. One important element of NoSQL databases is that they typically don’t enforce a strict schema. Data is distributed over partitions that can be scaled horizontally and is retrieved usi...

ID	Source	Page	Excerpt
KB-E009	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	**c. Event-Driven Communication:** Asynchronous event-driven patterns prove superior for e-commerce workloads, providing better scalability under high load conditions and improved fault tolerance [4][5]. The natural event-oriented nature of e-commerce business processes aligns well with event-driven architectures. **...
KB-E010	Rag Database/box2_domain/ecommerce_search_opensearch_aws.md	0	## Scaling for traffic surges and catalog growth *(PDF p. 11)* E-commerce platforms face unpredictable traffic surges and growing product catalogs. The article describes two broad scaling approaches: - **Vertical scaling:** increase the resources available to existing data nodes. - **Horizontal scaling:** add more...

Validation & Reviewer Findings

Overall Verdict: PASS

Refinement rounds: 2

No open findings were recorded on the accepted review.

Traceability

Features -> Decisions / Components

Feature	Name	ADRs	Components
FEAT-001	Product browsing and search	ADR-001, ADR-002, ADR-003, ADR-005	COMP-001, COMP-003, COMP-005, COMP-006, COMP-013, COMP-014, COMP-015
FEAT-002	Customer account management	ADR-001, ADR-002, ADR-005	COMP-003, COMP-004, COMP-013
FEAT-003	Shopping cart management	ADR-001, ADR-002, ADR-005	COMP-003, COMP-007, COMP-013
FEAT-004	Order placement and processing	ADR-001, ADR-002, ADR-003, ADR-005	COMP-003, COMP-008, COMP-011, COMP-013, COMP-016, COMP-017
FEAT-005	Payment handling	ADR-001, ADR-002, ADR-003, ADR-005	COMP-003, COMP-009, COMP-011, COMP-013, COMP-016, COMP-017
FEAT-006	Inventory and availability management	ADR-001, ADR-002, ADR-005	COMP-003, COMP-010, COMP-013
FEAT-007	Order status and history view	ADR-001, ADR-005	COMP-008
FEAT-008	Peak concurrency support	ADR-001, ADR-002, ADR-003, ADR-004, ADR-005	COMP-002, COMP-003, COMP-008, COMP-010, COMP-012, COMP-013
FEAT-009	High availability during peaks	ADR-001, ADR-002, ADR-003, ADR-004, ADR-005	COMP-002, COMP-003, COMP-008, COMP-010, COMP-012, COMP-013

Feature	Name	ADRs	Components
FEAT-010	Responsive customer-facing interactions	ADR-001, ADR-002, ADR-003, ADR-004, ADR-005	COMP-001, COMP-002, COMP-003, COMP-008, COMP-013, COMP-014
FEAT-011	Elastic scalability	ADR-001, ADR-002, ADR-003, ADR-004, ADR-005	COMP-002, COMP-003, COMP-008, COMP-012, COMP-013

Decision Topics -> ADRs

Topic	Question	ADRs
TOPIC-1	service decomposition and boundaries	ADR-001
TOPIC-2	data ownership and persistence strategy	ADR-002
TOPIC-3	integration style: synchronous vs asynchronous communication	ADR-003
TOPIC-4	scaling and availability strategy	ADR-004, ADR-005

ADRs -> Evidence

ADR	Evidence
ADR-001	KB-E004, KB-E001
ADR-002	KB-E005, KB-E008
ADR-003	KB-E002, KB-E009
ADR-004	KB-E003, KB-E006
ADR-005	KB-E006, KB-E010

Limitations / Open Items

No unresolved items are recorded on this run.